Protocol Design

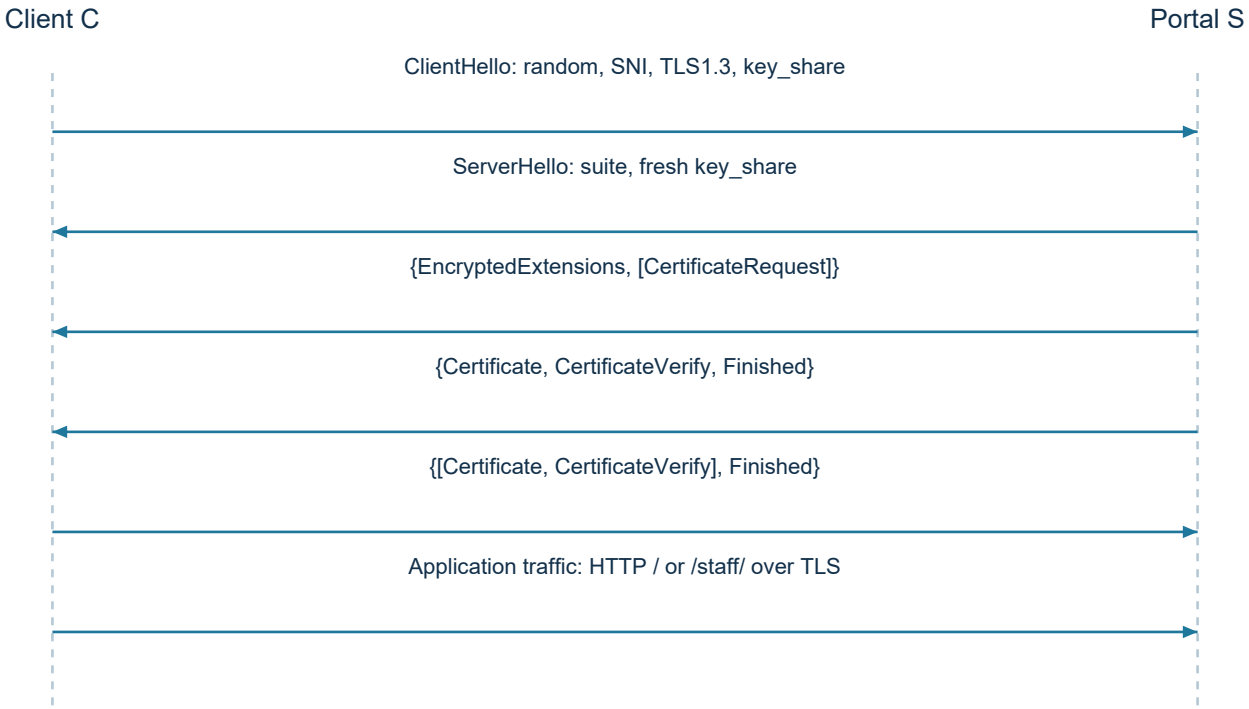
Group 03 | TLS and IKE designs; first Kerberos-style draft | Not implemented

Shared rules

Endpoints use Group 03 identities in Topology/PKI designs. Cryptographic primitives come from OpenSSL, strongSwan and pyca/cryptography, never hand-written. Later crypto tool choices: HMAC-SHA-256, Ed25519 signatures, AES-256-GCM AEAD and X25519 plus HKDF-SHA-256; separate keys by purpose and test known-answer vectors in M2. TLS/IKE use their native key schedules, not the custom protocol below.

TLS 1.3 full handshake as intended

Parties: C=client-br/public visitor; S=web-hq portal.g03.scp.lab. Arrows after ServerHello carry encrypted handshake records. CR=CertificateRequest. A public client may send an empty certificate list; staff must supply a valid chain and prove private-key possession. No 0-RTT; initially disable tickets/resumption so the tested path is an unambiguous full handshake.



ClientHello proposes TLS 1.3, AEAD suites, signature schemes, a fresh random and X25519 key share, SNI=portal.g03.scp.lab. ServerHello selects suite/group and fresh share/random. ECDHE plus HKDF yields handshake then application traffic keys. CertificateVerify signs the transcript; Finished authenticates it with a derived key. An optional HelloRetryRequest is a standard branch to test if group selection requires it; the primary diagram assumes matching groups.

S sends leaf plus Issuing CA. C checks Root trust, SAN hostname, validity, KU/serverAuth and intended revocation policy. C's CertificateVerify proves staff key possession; S validates chain, clientAuth, freshness/CRL and policy. The proposed TLS 1.3 suites are TLS_AES_128_GCM_SHA256 and TLS_AES_256_GCM_SHA384; TLS 1.2 compatibility uses ECDHE-ECDSA-AES128/256-GCM only. TLS below 1.2 and non-AEAD/non-PFS offers fail. Final HSTS is sent over HTTPS; public HTTP redirects except the pki CRL endpoint. These controls are future M2 work.

mTLS for /staff/

mTLS means both peers authenticate with certificates. HTTP paths are known only after the TLS handshake. Therefore nginx will request an OPTIONAL client certificate at the server level, then require ssl_client_verify=SUCCESS for both /staff and /staff/ (including descendants), with a generic denial otherwise. Do not attempt a path-specific TLS handshake or rely on TLS renegotiation. Public visitors without a certificate can still access /. An invalid certificate that is supplied can cause a connection-level failure even on a public URL; public access is guaranteed for visitors who supply none.

Proposed ssl_verify_depth=2 supports leaf -> Issuing -> Root; test the actual chain/depth semantics. ssl_client_certificate holds trusted CA material; ssl_crl holds current PEM CRLs. Record the verified subject/serial as identity; display escaped identity text only, never trust a client HTTP identity header. Normalize paths and test /staff, /staff/, encoded paths and traversal. No fallback to anonymous staff access. Authentication failures log diagnostic reason internally but return a generic page. CRL refresh, resumption policy and post-revocation reconnection tests follow the PKI plan.

IKEv2 message sequence

I=gw-branch.g03.scp.lab at 172.16.3.20; R=gw-hq.g03.scp.lab at 172.16.3.10. IKE_SA_INIT negotiates crypto and DH; IKE_AUTH authenticates identities/certificates and creates the first CHILD_SA. SK{...} denotes IKE-encrypted and integrity-protected payloads, not application TLS.



SAi1/SAr1 contain selected IKE proposals; KEi/KEr are ephemeral DH shares; Ni/Nr are fresh nonces. SKEYSEED and IKE SK_* keys derive from DH and nonces using the IKEv2 PRF; AUTH binds identities to the initial exchange and signed data. Message IDs/sequence windows and authenticated encryption protect subsequent exchanges from replay. CERTREQ requests acceptable CA; never accepts an arbitrary Root supplied by the peer.

Proposed IKE: aes256gcm16-prfsha384-ecp384; ESP: aes256gcm16-ecp384. Certificates ECDSA P-256 as profiled. Authentication is public-key only; no PSK. Pin remote identity exactly to gw-hq.g03.scp.lab or gw-branch.g03.scp.lab and require valid chain, SAN identity and revocation. TSi=10.3.2.0/24, TSr=10.3.1.0/24; reverse at HQ. Tunnel mode only. Do not broaden selectors to 0.0.0.0/0.

PFS means a compromised long-term key cannot decrypt recorded sessions. The initial CHILD_SA derives from the IKE exchange's DH. Subsequent CREATE_CHILD_SA rekeys must include fresh ECDH (ecp384), not merely claim separate initial CHILD DH. Proposed child rekey 45 min, hard lifetime 60 min; IKE rekey 3 h,

hard lifetime 4 h; DPD delay 30 s and bounded retry/timeout policy to be verified against strongSwan behavior. Exact proposal support and timers remain subject to runtime checks.

Final firewall permits WAN UDP 500/4500 and ESP only, drops unmatched inbound and clear LAN-to-LAN forwarding, and requires outbound/inbound IPsec policy for selectors. Bring tunnel up automatically only in M3. In M1 no swanctl.conf exists at the fixed gateway paths and no IKE service is launched; scoped HTTP baseline bypass is deliberate and not final hardening. Tunnel-loss and daemon-stop tests must prove fail-closed behavior later.

Kerberos-style draft: notation and keys

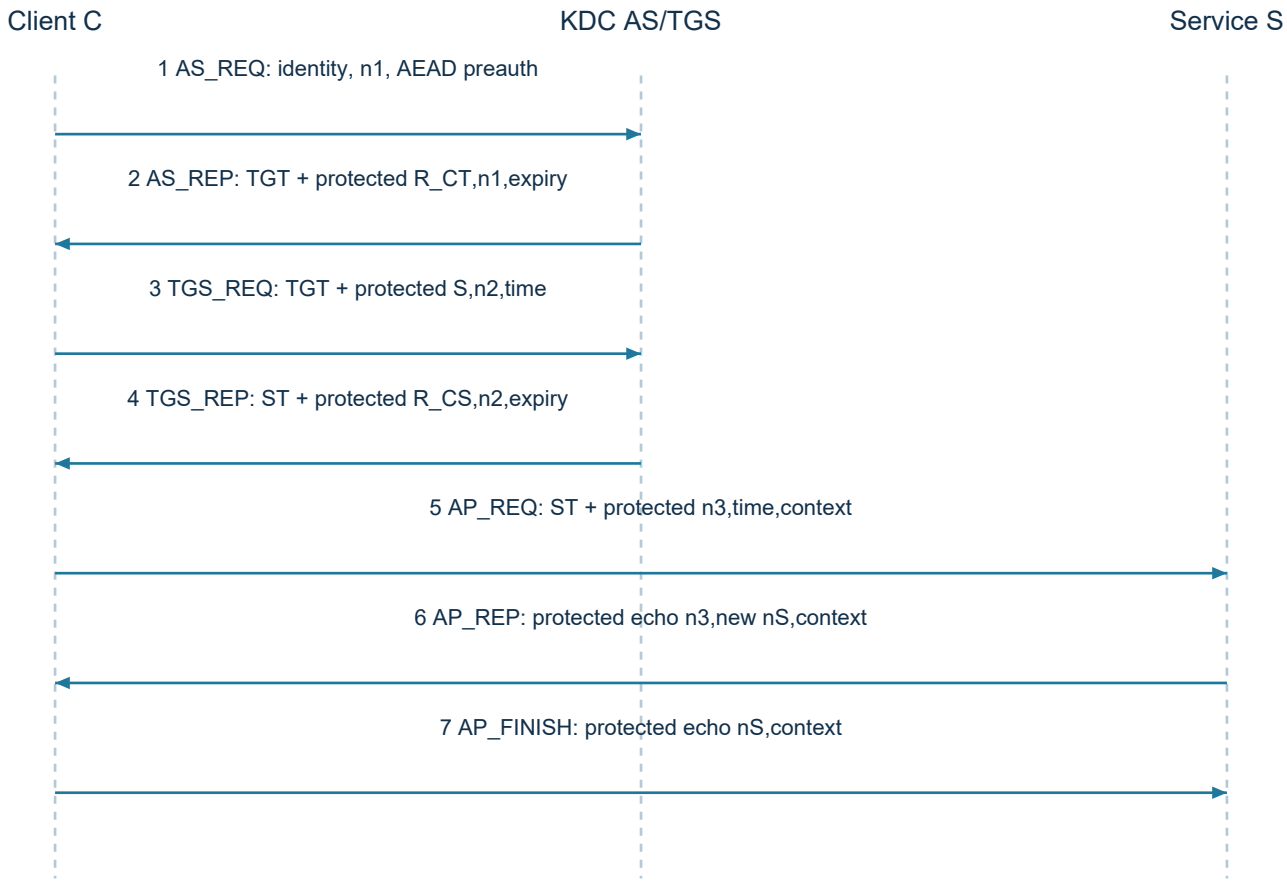
This is an educational custom protocol inspired by AS/TGS/AP, not interoperable Kerberos V5 and not a claim to implement RFC 4120. AS=Authentication Server and TGS=Ticket Granting Server, both roles on kdc TCP 8800. AP=Application exchange with service S on web-hq TCP 8801. C is a staff principal on client-br.

K_C is a unique per-principal long-term key; K_T is the KDC-only TGT-wrapping key; K_S is a service long-term key shared only by KDC and S. Provision them locally, outside Git; never transmit them. For the required wrong-password test, derive K_C with scrypt from a lab password and public per-user salt; return generic errors and rate-limit. Proposed scrypt N=32768,r=8,p=1 and 32-byte output require resource testing. Password-derived challenge responses can permit offline dictionary attacks if captured; use strong lab passphrases and document this residual limitation. No claim to a PAKE.

E_K[type,C,dst,fields] means AES-256-GCM with a unique 96-bit encryption nonce plus authenticated ciphertext. Associated data (AAD) binds protocol version, message type, sender, receiver and request ID using canonical length-prefixed encoding. Derive direction/type-specific AEAD keys from K using HKDF-SHA-256 and distinct labels; reject unknown versions/types/fields. Encryption nonces differ from challenge nonces n1/n2/n3. A proposed persistent nonce counter per key plus process-safe allocator avoids reuse; rotate keys before counter reset/overflow. Restart/recovery semantics must be implemented and tested before security claims.

R_CT and R_CS are fresh random 32-byte seeds for client/TGS and client/service session keys. K_CT=HKDF(R_CT, salt=tgt_id, info=version|C|TGS|expiry); K_CS=HKDF(R_CS, salt=st_id, info=version|C|S|expiry). Distinct derived keys protect authenticators and key-confirmation directions. IDs/nonces come from secrets; request challenge nonces are 128 bits. Time is UTC seconds; initial skew window +/-60 s, TGT lifetime 10 min and service lifetime 5 min capped by TGT. These are draft local choices, not spec constants.

AS/TGS/AP full draft sequence



1. C -> AS: version,C,n1,req1, E_KC[AS_PREAUTH,C,AS,{C,n1,req1,tC}]. AS validates principal and AEAD before accepting freshness; known/unknown principal errors are identical. Public salt provisioning does not disclose K_C. Do not insert unauthenticated IDs into the trusted replay cache.

2. AS -> C: TGT, E_KC[AS_REP,AS,C,{n1,req1,R_CT,tgt_id,issued,expires,TGS}]. TGT=E_KT[TGT,AS,TGS,{C,TGS,R_CT,tgt_id,issued,expires}]. C checks its challenge and derives K_CT. C cannot decrypt or modify TGT. AS records authenticated request replay until its freshness window expires.

3. C -> TGS: req2,S,TGT, E_KCT[TGS_AUTH,C,TGS,{C,S,tgt_id,req2,n2,tC2}]. TGS decrypts TGT, checks destination/lifetime and authenticated request identity, freshness and replay cache. Requested S must be authorized and bound in authenticator/AAD.

4. TGS -> C: ST, E_KCT[TGS_REP,TGS,C,{S,n2,req2,R_CS,st_id,issued,expires}]. ST=E_KS[ST,TGS,S,{C,S,R_CS,st_id,issued,expires}]. C checks n2, S and expiry then derives K_CS. S remains the only service able to open ST.

5. C -> S: req3,ST, E_KCS[AP_REQ,C,S,{C,S,st_id,req3,n3,tC3}]. S checks ticket, identities, expiry, time and atomic replay-cache insert keyed by st_id,C,req3,n3 AFTER authentication. Cache retention lasts ticket expiry plus skew; authenticate before application side effects.

6. S -> C: E_KCS[AP_REP,S,C,{st_id,req3,n3,nS,tS,context}]. Echo n3 proves S possesses K_CS; bind session context and new server challenge nS. C validates before considering S authenticated.

7. C -> S: E_KCS[AP_FINISH,C,S,{st_id,req3,nS,context}]. S consumes a pending challenge once and marks the session established. Neither side grants application actions until mutual confirmation. Both derive application keys with a separate HKDF label/context; use monotonic sequence numbers and replay windows for later application records.

Replay, errors and draft validation

Reject duplicate valid AS/TGS/AP requests atomically; parallel duplicate delivery must not bypass the cache. Cache state persists through restart or invalidate all active tickets by rotating wrapping/session state; document the chosen recovery policy. Accept no authenticator outside skew or ticket validity. Synchronize lab clocks from trusted host, detect large jumps, and fail rather than silently enlarge windows. Expired tickets and stale challenges cannot establish sessions. Rate-limit and bound pending/cache state without evicting still-live replay entries; refuse new sessions when capacity is exhausted.

Wrong AEAD tag, wrong key, wrong destination, changed ticket ID, stale time, changed nonce and duplicate AP_FINISH must be refused with generic errors and sanitized logs. Planned weak no-freshness variant lives only under attacks/replay/ in M3. Test T12a on weak variant then T12b using the same message on fixed variant. M1 supplies design only; framing (bounded length-prefixed messages, initial 64 KiB cap), nonce persistence, scrypt performance, key lifecycle, clock recovery and strongSwan EKU compatibility remain draft decisions for later tests.

References

Assignment sections 6-10. TLS sequence: RFC 8446 sections 2 and 4, https://www.rfc-editor.org/rfc/rfc8446 . IKE exchanges/key derivation: RFC 7296 sections 1-2, https://www.rfc-editor.org/rfc/rfc7296 . Kerberos vocabulary: RFC 4120, https://www.rfc-editor.org/rfc/rfc4120 . nginx certificate/depth/CRL directives: https://nginx.org/en/docs/http/ngx_http_ssl_module.html . strongSwan proposal and identity options: https://docs.strongswan.org/docs/latest/swanctl/swanctlConf.html . Version-specific behavior must be checked on the Ubuntu image later.